ShopEasy Engineering: Past Incident Reports

Internal postmortem archive. Each section describes one resolved incident: what happened, why, how it was fixed and what we learned.

INC-101: checkout returns 500 when discount is missing

Severity	Sev1	Date	2026-03-14	Status	Resolved

Summary

Customers placing orders without a coupon received HTTP 500 at the checkout confirmation step. Roughly 22% of checkout attempts failed for 41 minutes.

Root cause

calculate_total assumed order.discount always exists. Orders with no discount raised a TypeError because discount_id was None and the code tried to read the amount of a missing object.

Fix applied

Added a guard clause that treats a missing discount as zero, plus a regression test covering orders with no coupon. Hotfix deployed after approval.

Lessons learned

Null handling in the pricing module needs tests for every optional field. Code review should flag removed null checks.

INC-087: payment page slow because external gateway timeout not handled

Severity	Sev2	Date	2026-08-18	Status	Resolved

Summary

The payment page stayed on a loading spinner for up to 30 seconds and then failed. Payment success rate dropped from 97% to 71%.

Root cause

The stripe gateway was responding slowly. The payment service waited with no retry policy, so every slow response became a timeout error and a failed payment.

Fix applied

Added an explicit timeout, retry with exponential backoff and a circuit breaker around the gateway client. Added an alert on gateway latency.

Lessons learned

External calls must always have timeouts and retries. Gateway latency should be monitored separately from our own latency.

INC-066: user could view another user's order

Severity	Sev1	Date	2026-06-30	Status	Resolved

Summary

A customer found that changing the order id in the URL showed another customer's order. 14 orders were viewed before the report.

Root cause

The get_order endpoint returned the order for any valid id. There was no ownership check, so authorization depended only on the user being logged in.

Fix applied

Added an authorization check that compares the order owner with the requesting user and returns 403 otherwise. Added audit logging for denied access.

Lessons learned

Every object endpoint needs an ownership check. Add an automated test that requests another user's resource.

INC-094: order total does not match sum of line items

Severity	Sev2	Date	2026-09-09	Status	Resolved

Summary

A database check constraint rejected some orders because the stored total differed from the sum of line items by one paisa.

Root cause

Line item prices were computed with floating point arithmetic and rounded at different places, so the total and the line items disagreed for some quantities.

Fix applied

Switched price arithmetic to decimal, rounded once per line item and recomputed the total from rounded lines.

Lessons learned

Never use floating point for money. Keep one rounding rule in one place.

INC-112: negative stock during flash sale

Severity	Sev3	Date	2026-09-21	Status	Resolved

Summary

Inventory showed negative available stock for popular items and some orders could not be fulfilled.

Root cause

Concurrent reservations updated the same inventory row without a lock, so two requests both read the same available quantity and both subtracted from it.

Fix applied

Used row-level locking (SELECT FOR UPDATE) in reserve_stock and added a stock never below zero constraint.

Lessons learned

Concurrency bugs only appear under load. Add a load test for flash sale scenarios.

Appendix: Developer Setup Guide

Welcome to the ShopEasy codebase. This guide explains how to set up a local environment for the checkout, payment, order, auth and inventory services. Follow the steps in order and ask in the team channel if anything is unclear.

Prerequisites. Install Python 3.12, Node 20, Docker and docker compose. Create a virtual environment and install the requirements file. Copy the example environment file and fill in the local database settings.

Services. The checkout service handles the cart and order confirmation. The payment service talks to the payment gateway. The order service stores orders and line items. The auth service manages login and tokens. The inventory service tracks stock. The notification service sends email.

Database. Start the local database with docker compose and run the migrations. Seed data includes sample products, a few users and example orders. If a migration fails, check the error message in the console and the database logs.

Logging. All services write logs to the logs folder. Set the log level to DEBUG locally. In staging and production the level is INFO and error logs are collected centrally. When debugging an error, look first at the most recent error entries.

Configuration. Timeout settings for outgoing calls are read from the environment. The default timeout is 30 seconds for the gateway and 5 seconds for internal services. Discount configuration lives in the discounts settings file and can be reloaded without a restart.

Testing. Run the unit tests with pytest and the frontend tests with the test runner. Add a test whenever you fix a bug. Pull requests need one review and a passing build before merging. If the build fails because of a flaky test, rerun it once and report it.

Order flow. A customer adds items to the cart, goes to checkout, optionally applies a discount, pays and receives a confirmation. Each order has a total which must equal the sum of its line items minus any discount. Users can only see their own orders.

Deployment. Merging to main triggers the pipeline. After tests pass the checkout service is deployed with a rolling restart. Check the health endpoint after every deploy and watch the error rate dashboard for the first ten minutes. Rollback instructions are in the runbook.

Getting help. If you see an error you do not understand, search the incident archive first, then ask a teammate. Please keep tickets short and include the order number, the time and the service name.